UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO



FACULTAD DE INGENIERÍA



MODELADOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

REPORTE DE CÓDIGO: PRÁCTICA 8

REPORTE DE PRÁCTICA: SISTEMA DE SENSORES Y ACTUADORES

ALUMNO: MIGUEL ANGEL MEDINA REYES

NÚMERO DE CUENTA: 322287123

PROFESOR: OSCAR MANUEL RUIZ HURTADO

SEMESTRE: 2027-1

1. Introducción

La presente práctica consiste en el diseño y simulación de un sistema de monitoreo industrial o de automatización (Internet de las Cosas - IoT) mediante el paradigma de Programación Orientada a Objetos (POO) en Java. El problema que se busca resolver es la recolección de datos de diferentes tipos de sensores (peso, presencia, temperatura) y la automatización de la respuesta de diversos actuadores (alarmas, bandas transportadoras, semáforos) ante situaciones de alerta. Se emplearon conceptos fundamentales de POO como herencia, polimorfismo, abstracción y encapsulamiento.

2. Objetivo

Desarrollar un programa utilizando programación orientada a objetos en Java, aplicando jerarquías de clases, clases abstractas, herencia y polimorfismo para modelar el comportamiento interactivo entre sensores y actuadores gestionados por una estación central.

3. Desarrollo del programa

3.1 Descripción del funcionamiento general

El sistema está centralizado en la clase Estacion, la cual gestiona arreglos de dispositivos (sensores y actuadores). El ciclo de funcionamiento recorre todos los sensores instalados para extraer sus lecturas; si alguno registra valores fuera de los límites definidos (generando una alerta), el sistema propaga este estado booleano a todos los actuadores, los cuales modifican su comportamiento (encendiéndose, deteniéndose o cambiando de color) en respuesta a la alerta.

3.2 Explicación de las clases utilizadas y relación entre ellas

Dispositivo (Clase Abstracta): Es la base de la jerarquía. Representa cualquier elemento del sistema, identificándolo con un ID y asociándole una Bateria mediante composición. Bateria: Gestiona la carga de energía (0-100%). Encapsula la lógica de descarga. Sensor (Clase Abstracta): Hereda de Dispositivo. Introduce el concepto de medición y alerta. Define métodos abstractos (medir() y hayAlerta()) que sus clases hijas deben implementar.

Actuador (Clase Abstracta): Hereda de Dispositivo. Mantiene un estado de encendido/apagado y define el método abstracto reaccionar(boolean alerta). Clases Concretas de Sensores (SensorPeso, SensorPresencia, SensorTemperatura): Heredan de Sensor. Implementan las reglas de negocio específicas para calcular sus valores reales y determinar en qué punto sus lecturas se consideran alertas.

Clases Concretas de Actuadores (Alarma, BandaTransportadora, Semaforo): Heredan de Actuador. Definen cómo se comporta el equipo. Por ejemplo, la BandaTransportadora gasta batería al avanzar y se detiene en alertas.

Estacion: Actúa como el controlador principal mediante la agregación de sensores y actuadores.

3.3 Atributos y Métodos importantes

La encapsulación asegura que atributos como carga, limite o recorrido sean privados, modificables solo a través de métodos controlados. El método más relevante es el método plantilla leer() en Sensor, que primero gasta energía llamando al método protegido gastar(), luego obtiene la medición con medir() (polimórfico) y evalúa hayAlerta() (polimórfico) para retornar un String formateado.

4. Código fuente

Se muestran los fragmentos más importantes que evidencian la implementación del polimorfismo y herencia. (Nota: El programa completo incluye todos los archivos del paquete dispositivos, sensores, actuadores y estacion).

```
// Fragmento de la clase abstracta Sensor demostrando métodos abstractos public abstract class
Sensor extends Dispositivo {
        private final String unidad;
        private double senal;

        public Sensor(String id, String unidad, int carga) {
                super(id, carga);
                this.unidad = unidad;
        }

        public abstract double medir(); // Implementado por cada subclase public abstract
        boolean hayAlerta();

        // Método que utiliza polimorfismo dinámico
        public final String leer() {
                gastar(1); // Gasta batería heredada de Dispositivo
                String lectura = getId() + " mide " + medir() + " " + unidad;
                if (hayAlerta()) {
                        lectura += " ALERTA";
                }
                return lectura;
        }
}
```



```
// Fragmento de la clase Estacion (Controlador central)
public void ciclo() {
```

```
        ciclos++;
        System.out.println("== Ciclo " + ciclos + " en " + nombre);
        boolean hayAlerta = false;

        // Evalúa alertas de forma polimórfica
        for (int i = 0; i < numSensores; i++) {
                System.out.println(sensores[i].leer());
                if (sensores[i].hayAlerta()) {
                        hayAlerta = true;
                }
        }

        // Dispara reacciones según el estado general
        for (int i = 0; i < numActuadores; i++) {
                System.out.println(actuadores[i].reaccionar(hayAlerta));
        }
}
```

6. Resultados

El programa cumplió exitosamente con su objetivo. La integración entre sensores y actuadores operó correctamente; se comprobó que una alerta disparada por una medición fuera de rango (como un sensor de presencia midiendo menos de 2 cm) propaga inmediatamente el estado de alerta a los actuadores en el mismo ciclo, activando las alarmas correspondientes y deteniendo las bandas transportadoras. La gestión de batería también funcionó de forma esperada al descontar puntos por cada ciclo operativo.

7. Conclusiones

A lo largo de esta práctica, se consolidó la comprensión sobre la importancia de utilizar clases abstractas para definir "contratos" o plantillas de comportamiento en la programación. La dificultad principal recayó en asegurar la correcta delegación de constructores (uso de super() y this()) entre clases hijas y padres. Se aprendió que el polimorfismo simplifica drásticamente el código de las clases administradoras como Estacion, ya que puede tratar con todos los dispositivos de manera uniforme sin preocuparse por los detalles de implementación específicos de cada sensor o actuador.